

Tarea 2: OpenCL y CUDA

Computación en GPU

Integrante: Matías Saavedra
Profesora: Nancy Hitschfeld K.
Auxiliar: Gustavo Medel
Ayudante: Catalina Gajardo

Fecha de entrega: 6 de junio de 2026
Santiago de Chile

Resumen

Este informe describe el desarrollo y la evaluación de implementaciones del Juego de la Vida de Conway en CPU (secuencial y paralela) y GPU, con el fin de medir las células evaluadas por segundo y el *speed-up* de la GPU respecto de la CPU. Se utiliza una variación de las reglas estándar en la que una célula muerta también nace si tiene exactamente 6 vecinos vivos (la regla *HighLife*, **B36/S23**). El núcleo del proyecto es agnóstico al backend: una grilla plana común, una única regla compartida y dos interfaces intercambiables (motor de simulación y *renderer*). A la fecha están implementados y verificados los motores de CPU y CUDA; este último se compara bit a bit contra la CPU como oráculo. Se comparan dos variaciones de configuración del kernel (tamaño de bloque y uso de memoria compartida) y se analiza a partir de qué tamaño de cuadrícula $N \times M$ conviene usar la GPU.

Índice de Contenidos

1. Introducción	1
1.1. Resultados esperados	1
2. Implementación	1
2.1. Arquitectura general	1
2.2. Implementación en CPU (secuencial y paralela)	3
2.3. Implementación paralela en CUDA	4
2.4. Implementación paralela en OpenCL	5
2.5. Variaciones de configuración	6
2.6. Verificación de correctitud	6
2.7. Metodología de medición	6
2.8. Ejecutables y flujo de uso	7
3. Resultados	7
3.1. Configuración experimental	7
3.2. CPU vs GPU por tamaño de cuadrícula	7
3.3. Resultados de las opciones elegidas	7
4. Análisis de resultados	7
4.1. Speed-up	8
4.2. Umbral de conveniencia GPU	8
4.3. Perfilamiento (Nsight / Chrono)	8
5. Conclusiones	8

Índice de Figuras

1. Diagrama de clases del proyecto. La Application posee un ISimEngine y un IRenderer (las dos estrategias). CpuEngine y CudaEngine realizan ISimEngine ; OpenCLEngine está planificado. Las flechas punteadas son dependencias: CpuEngine y CudaEngine usan ambos LifeRules::nextState	2
---	---

Índice de Códigos

1. include/gol/LifeRules.hpp — la regla variante, compartida por CPU y CUDA.	2
2. src/engines/cpu/CpuEngine.cpp — un paso (mismo código para secuencial y paralelo).	3
3. src/engines/cuda/kernel.cu — kernel de memoria global (un hilo por celda). .	4
4. src/engines/cuda/kernel.cu — mapeo de bloque y elección de kernel.	5

5.	src/engines/cuda/CudaEngine.cu — un paso con cronometraje por eventos. . .	5
----	--	---

1. Introducción

El Juego de la Vida de Conway es un autómata celular bidimensional en el que cada celda de una grilla $N \times M$ se encuentra viva o muerta y evoluciona en pasos discretos (generaciones) según el estado de sus ocho vecinos. En esta tarea se considera la siguiente *variación* de las reglas estándar:

- Una célula nace en un espacio muerto si tiene exactamente 3 vecinos vivos.
- Una célula sobrevive a la próxima generación si tiene 2 o 3 vecinos vivos.
- Una célula también nace en un espacio muerto si tiene exactamente 6 vecinos vivos.

La cláusula adicional (nacer con 6 vecinos) corresponde a la regla conocida como *HighLife* (B36/S23). Es el detalle más fácil de implementar de forma inconsistente, por lo que debe valer idénticamente en todos los backends.

El objetivo es implementar el algoritmo en CPU (secuencial), CUDA y OpenCL, medir su rendimiento en *células evaluadas por segundo*, comparar dos técnicas de optimización del kernel y analizar el *speed-up* respecto de la versión secuencial, determinando desde qué tamaño de grilla conviene usar la GPU.

1.1. Resultados esperados

El problema es un *stencil* de 9 puntos sobre celdas de 1 byte: muy poco cálculo por byte leído, es decir, limitado por ancho de banda de memoria. Se espera, en consecuencia:

- Que la GPU supere ampliamente a la CPU para grillas grandes, donde su mayor ancho de banda y paralelismo dominan frente al costo de transferencia *host* $\leftrightarrow$ *device*.
- Que para grillas pequeñas el costo de lanzamiento del kernel y de transferencia haga competitiva (o superior) a la CPU.
- Que el tamaño de bloque dé una curva cóncava que se estabiliza hacia 128–256 hilos, y que la memoria compartida aporte poco o nada, precisamente por tratarse de un *stencil* de 1 byte que ya reside en caché.

2. Implementación

2.1. Arquitectura general

El proyecto sigue el patrón *estrategia*: un núcleo agnóstico al backend más dos interfaces intercambiables. Lo único que difiere realmente entre backends — el cómputo de una generación — vive tras **ISimEngine**; la salida vive tras **IRenderer**; y la aplicación arma en tiempo

de ejecución un motor y un *renderer* a partir de la configuración. La Figura 1 resume las clases y relaciones.

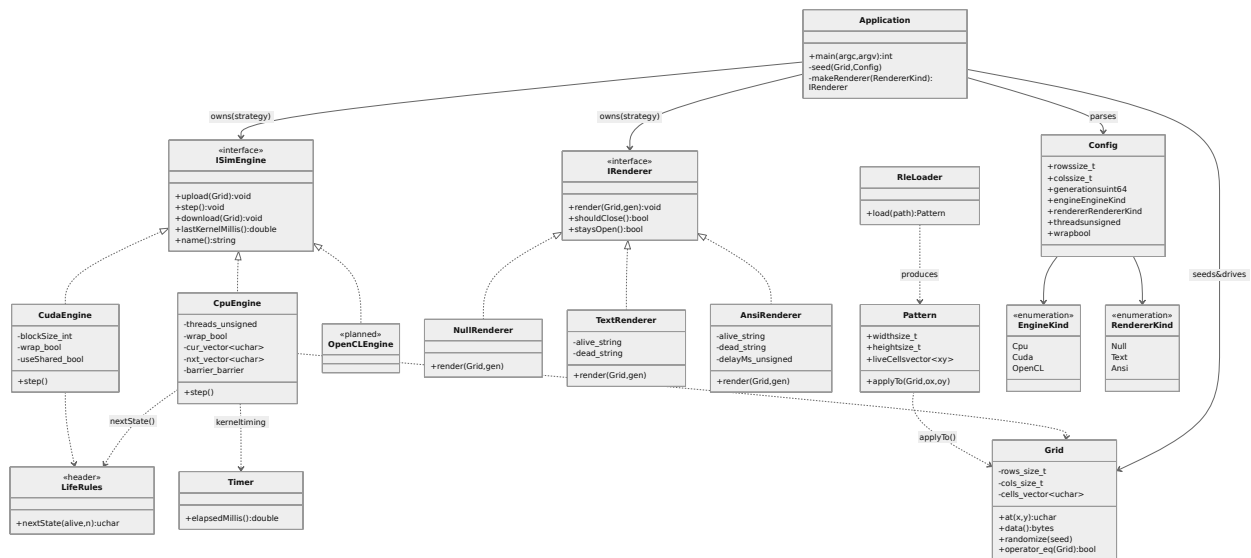


Figura 1: Diagrama de clases del proyecto. La **Application** posee un **ISimEngine** y un **IRenderer** (las dos estrategias). **CpuEngine** y **CudaEngine** realizan **ISimEngine**; **OpenCLEngine** está planificado. Las flechas punteadas son dependencias: **CpuEngine** y **CudaEngine** usan ambos **LifeRules::nextState**.

Tres decisiones del núcleo hacen posible comparar los backends de forma justa:

Una grilla plana compartida.

Grid es un `std::vector<unsigned char>` contiguo, fila por fila, indexado $y \cdot \text{cols} + x$ (0 = muerta, 1 = viva). Es exactamente el formato de un buffer de *device*, de modo que subir/bajar la grilla es una copia sin traducción y los tres backends operan sobre los mismos bytes. Se acepta el costo de 1 byte por celda (en vez de empaquetar bits) para mantener el *stencil* como una simple lectura de arreglo, idéntica en CPU y GPU.

Una única regla, compilada para *host* y *device*.

La regla variante se escribe una sola vez en **LifeRules.hpp**. Bajo **nvcc** se califica `__host__` `__device__`, de modo que la CPU y CUDA usan el mismo código fuente (Código 1); OpenCL no puede incluir encabezados de C++, por lo que replicará esta función en **kernel.cl**.

Código 1: `include/gol/LifeRules.hpp` — la regla variante, compartida por CPU y CUDA.

```

1 #ifndef __CUDACC__
2 #define GOL_FN __host__ __device__ inline
3 #else
4 #define GOL_FN inline

```

```

5 #endif
6
7 GOL_FN unsigned char nextState(unsigned char alive, int neighbors) {
8     if (alive)
9         return (neighbors == 2 || neighbors == 3) ? 1u : 0u; // sobrevive
10    return (neighbors == 3 || neighbors == 6) ? 1u : 0u; // nace (3 o 6)
11 }

```

Manejo de bordes idéntico.

Por defecto los bordes son acotados (un vecino fuera de rango cuenta como muerto); con `-wrap` se vuelven toroidales. Ambos modos deben coincidir bit a bit entre backends, y se verifican en las pruebas (Sección 2.6).

Contrato de doble buffer (*ping-pong*).

Cada motor mantiene dos buffers; `step()` lee el “actual”, escribe el “siguiente” y los intercambia, de modo que una llamada avanza exactamente una generación y nunca solapa entrada y salida. Las GPU hacen el intercambio en el *device*; la grilla del *host* solo se toca en `upload()/download()`.

2.2. Implementación en CPU (secuencial y paralela)

Una sola clase, `CpuEngine`, sirve tanto al *baseline* secuencial como a la versión paralela, porque el paso del Juego de la Vida no tiene dependencias intra-generación: cada celda lee solo el buffer actual y escribe el siguiente. Paralelizar es, entonces, repartir las *filas* entre hilos; la regla por celda es idéntica. El recuento de vecinos honra el modo de borde y suma los valores 0/1 de los vecinos; la regla compartida (`nextState`) decide el estado siguiente.

El reparto es por bandas de filas contiguas (buen comportamiento de caché y escrituras disjuntas). Con `threads == 1` no se crea ningún hilo ni barrera: el barrido secuencial es el caso degenerado de una sola partición. Con `threads >= 2` se usa un *pool* de hilos persistente sincronizado con una `std::barrier` de dos fases, de modo que el costo de crear hilos no contamina la métrica. El valor `threads == 0` se resuelve a `std::thread::hardware_concurrency()`.

Código 2: `src/engines/cpu/CpuEngine.cpp` — un paso (mismo código para secuencial y paralelo).

```

1 void CpuEngine::step() {
2     Timer t;
3     if (workers_.empty()) { // secuencial: un solo barrido, sin sincronización
4         computeRows(cur_.data(), nxt_.data(), 0, rows_);
5     } else { // paralelo: bandas de filas
6         src_ = cur_.data();
7         dst_ = nxt_.data();
8         barrier_>arrive_and_wait(); // libera a los workers sobre sus filas
9         auto [yBegin, yEnd] = rangeFor(0);
10        computeRows(src_, dst_, yBegin, yEnd); // el hilo main calcula la partición 0
11        barrier_>arrive_and_wait(); // espera a que todas las particiones terminen

```

```

12 }
13 std::swap(cur_, nxt_);           // ping-pong
14 lastMs_ = t.elapsedMillis();    // cronometraje con std::chrono
15 }

```

Como secuencial y paralelo ejecutan la misma función de cómputo (`computeRows`) y solo difieren en el rango de filas, producen tableros idénticos; esto se comprueba en las pruebas (Sección 2.6).

2.3. Implementación paralela en CUDA

`CudaEngine` usa un hilo por celda sobre una grilla 2D de bloques. Posee dos buffers de *device* y realiza el *ping-pong* en el *device*; la grilla del *host* solo se toca en `upload` (copia H2D) y `download` (D2H). El encabezado del motor no incluye tipos de CUDA (los manejadores se guardan como punteros opacos), de modo que `main.cpp` compila con el compilador anfitrión y solo los `.cu` pasan por `nvcc`.

Hay dos kernels seleccionables en tiempo de ejecución. El de memoria global lee los ocho vecinos directamente; reutiliza `nextState` de `LifeRules.hpp` y replica el manejo de bordes de la CPU (Código 3).

Código 3: `src/engines/cuda/kernel.cu` — kernel de memoria global (un hilo por celda).

```

1  __global__ void lifeGlobal(const unsigned char* src, unsigned char* dst,
2                          int rows, int cols, bool wrap) {
3      const int col = blockIdx.x * blockDim.x + threadIdx.x;
4      const int row = blockIdx.y * blockDim.y + threadIdx.y;
5      if (col >= cols || row >= rows) return;
6      int n = 0;
7      for (int dy = -1; dy <= 1; ++dy)
8          for (int dx = -1; dx <= 1; ++dx) {
9              if (dx == 0 && dy == 0) continue;
10             int nc = col + dx, nr = row + dy;
11             if (wrap) { nc = (nc + cols) % cols; nr = (nr + rows) % rows; }
12             else if (nc < 0 || nr < 0 || nc >= cols || nr >= rows) continue; // borde: muerto
13             n += src[(size_t)nr * cols + nc];
14         }
15     const size_t idx = (size_t)row * cols + col;
16     dst[idx] = nextState(src[idx], n); // misma regla que la CPU
17 }

```

El lanzador mapea el tamaño de bloque sobre un *tile* 2D de ancho fijo 32 y elige el kernel. El ancho 32 alinea los accesos: hilos consecutivos de un *warp* leen celdas (de 1 byte) consecutivas de una fila, logrando lecturas coalescentes, lo correcto para un *stencil* limitado por memoria.

Código 4: src/engines/cuda/kernel.cu — mapeo de bloque y elección de kernel.

```

1 const int bx = 32; // filas de 32 -> lecturas coalescentes
2 const int by = (blockSize + bx - 1) / bx; // 32/64/128/256 -> by = 1/2/4/8
3 dim3 block(bx, by);
4 dim3 grid((cols + bx - 1) / bx, (rows + by - 1) / by);
5 if (useShared)
6     lifeShared<<<grid, block, (bx + 2) * (by + 2)>>>(src, dst, rows, cols, wrap);
7 else
8     lifeGlobal<<<grid, block>>>(src, dst, rows, cols, wrap);

```

El kernel de memoria compartida (`-shared`) primero carga cooperativamente un *tile* de $(bx+2) \times (by+2)$ celdas (las del bloque más un halo de una celda) en memoria compartida —aplicando el mismo manejo de borde durante la carga del halo— y luego lee los vecinos desde el *tile*. Ambos kernels producen resultados idénticos al oráculo de CPU.

El cómputo se cronometra con `cudaEvent_t` alrededor del lanzamiento, de modo que `lastKernelMillis()` reporta solo el tiempo del kernel, excluyendo las transferencias; es el análogo en GPU del cronometraje con `std::chrono` de la CPU.

Código 5: src/engines/cuda/CudaEngine.cu — un paso con cronometraje por eventos.

```

1 void CudaEngine::step() {
2     cudaEventRecord(start);
3     launchLifeStep(dCur_, dNxt_, rows_, cols_, wrap_, blockSize_, useShared_);
4     cudaEventRecord(stop);
5     cudaEventSynchronize(stop);
6     cudaEventElapsedTime(&ms, start, stop); // solo el kernel, sin H2D/D2H
7     lastMs_ = ms;
8     std::swap(dCur_, dNxt_); // ping-pong en el device
9 }

```

El *target* de compilación es `sm_75` (la GPU de pruebas, una GTX 1660 Ti), con `-lineinfo` para que Nsight Compute pueda atribuir tiempo a líneas de código fuente.

2.4. Implementación paralela en OpenCL

Pendiente. El motor de OpenCL (`OpenCLEngine`) aún no está implementado. Seguirá el mismo diseño que CUDA: dos buffers de *device*, *ping-pong* en el *device*, los mismos dos kernels (global y con memoria local) y los mismos botones `-block/-shared`. La regla y el manejo de borde se replicarán en un `kernel.cl` en C plano, leído en tiempo de ejecución y mantenido diff-eable contra `LifeRules.hpp`. El cronometraje del kernel usará eventos de la cola de comandos con `CL_QUEUE_PROFILING_ENABLE`. Se verificará bit a bit contra el oráculo de CPU, igual que CUDA.

2.5. Variaciones de configuración

De las tres opciones del enunciado se eligieron dos:

1. **Tamaño de bloque** $\in \{32, 64, 128, 256\}$ (botón `-block`).
2. **Memoria local por bloque** *vs.* sin memoria local (botón `-shared`).

No se eligió la opción de arreglos 2D *vs.* mapeo a 1D porque el diseño fija una grilla plana 1D compartida por todos los backends; usar arreglos 2D iría en contra de esa decisión arquitectónica. Ambas variaciones son seleccionables en tiempo de ejecución, de modo que un único binario ejecuta toda la matriz de experimentos.

2.6. Verificación de correctitud

`CpuEngine` es el oráculo de referencia. Las pruebas (GoogleTest) verifican, en orden de dependencia:

- **Regla variante en CPU:** un bloque permanece estático, un *blinker* oscila con período 2 y una célula muerta con exactamente 6 vecinos nace (el caso que define la variante).
- **Equivalencia secuencial *vs.* paralela:** misma semilla y mismas generaciones producen un tablero idéntico bit a bit para distinto número de hilos y ambos modos de borde, incluso en grillas no divisibles.
- **Equivalencia entre backends (CUDA):** `cuda_equivalence_test.cpp` siembra CUDA y CPU igual y exige igualdad bit a bit contra el oráculo para los tamaños de bloque $\{32, 64, 128, 256\}$, kernels global/compartido y ambos modos de borde. OpenCL seguirá el mismo patrón.

Adicionalmente, el runner `bgolly` de Golly (grilla infinita) simula exactamente B36/S23 y sirve como oráculo externo independiente.

2.7. Metodología de medición

La métrica principal es *células evaluadas por segundo*:

$$\text{Mcélulas/s} = \frac{N \cdot M \cdot G}{t_{\text{transcurrido}} \times 10^6}, \quad (1)$$

con G el número de generaciones. El cronometraje es uniforme y en programa (no desde un *profiler*), expuesto por `ISimEngine::lastKernelMillis()`: `std::chrono` en CPU, `cudaEvent_t` en CUDA y eventos de cola en OpenCL. Se reporta el tiempo de kernel por separado del de transferencias, y se mide siempre con el `NullRenderer` para que el costo de salida y de transferencia no contamine la métrica. El *speed-up* es

$$S(N) = \frac{T_{\text{CPU}}(N)}{T_{\text{GPU}}(N)}. \quad (2)$$

Para los barridos, el binario ofrece un modo CSV: `-csv` imprime una fila por corrida (columnas `backend`, `rows`, `cols`, `generations`, `threads`, `block`, `shared`, `wrap`, `kernel_ms`, `wall_ms`, `mcells_kernel`, `mcells_wall`) y `-csv-header` imprime solo el encabezado; así un *script* escribe el encabezado una vez y agrega una fila por configuración, lista para Pandas.

2.8. Ejecutables y flujo de uso

El proyecto compila un único ejecutable `gol` que arma en tiempo de ejecución el motor (`-engine cpu|cuda|opencl`) y el *renderer* (`-renderer null|text|ansi`) según la línea de comandos, siembra la grilla (con una semilla aleatoria determinista o un patrón RLE) y ejecuta el bucle de generaciones. Bajo `NullRenderer` el bucle es solo `step()`, sin transferencias ni dibujado, de modo que la medición no se contamina. Los motores de GPU son opcionales en CMake (`-DBUILD_CUDA=ON`); el binario funciona en una máquina sin CUDA ni OpenCL.

3. Resultados

3.1. Configuración experimental

Las mediciones se realizan en un equipo con procesador Intel Core i7-9750H (6 núcleos físicos / 12 hilos, caché L2 de 1,5 MiB y L3 de 12 MiB) y GPU NVIDIA GeForce GTX 1660 Ti (Turing, capacidad de cómputo 7.5), bajo Linux, con CUDA y compilación en C++23. El motor de CPU se mide con 1 hilo (*baseline* secuencial) y con todos los hilos disponibles; el de CUDA con los cuatro tamaños de bloque y con/sin memoria compartida.

Se mide el número de células evaluadas por segundo (Ecuación 1) para grillas cuadradas $N \in \{128, 256, 512, 1024, 2048, 4096\}$, con un número fijo de generaciones por configuración y promediando varias corridas independientes (tomando el mejor tiempo para reducir el ruido de *turbo* y contención). Todas las mediciones usan el `NullRenderer`.

3.2. CPU vs GPU por tamaño de cuadrícula

Pendiente: ejecutar el barrido y presentar aquí la tabla y los gráficos de Mcélulas/s en función de N para CPU (1 hilo y multi-hilo) y CUDA (mejor configuración).

3.3. Resultados de las opciones elegidas

Pendiente: curvas de Mcélulas/s por tamaño de bloque $\{32, 64, 128, 256\}$ y comparación con/sin memoria compartida, para los N grandes.

4. Análisis de resultados

4.1. Speed-up

Pendiente: reportar $S(N)$ (Ecuación 2) por configuración a partir de los datos de la Sección 3.

4.2. Umbral de conveniencia GPU

Pendiente: identificar el tamaño de grilla a partir del cual $S(N) > 1$ de manera sostenida, considerando el costo de transferencia en grillas pequeñas.

4.3. Perfilamiento (Nsight / Chrono)

Pendiente: hallazgos del profiling de la mejor solución CUDA con Nsight Systems (línea de tiempo) y Nsight Compute (métricas internas del kernel: ocupación, stalls, throughput de memoria), y de la mejor solución OpenCL con `std::chrono`.

5. Conclusiones

Pendiente: resumir los principales hallazgos (speed-up alcanzado, umbral de conveniencia, efecto de las dos variaciones de configuración) y las limitaciones encontradas.